University Of Pisa
Computer Science Department
Academic Year 2025/2026

ML 2025 PROJECT

Type B

Authors: **Salmi, Chafai, Yakoubi**
Team: **SYC**

OBJECTIVES

Compare the performance of 4 learning algorithms **(Linear, NN, SVM, KNN)** on:

- MONK datasets → **Binary classification**
- ML-CUP25 dataset → **Multi-target regression** (4 outputs)

Our aim is to explore models with different complexity (simple vs complex) and compare their generalization capacity on different datasets.

IMPLEMENTATION

Implemented in **Python**. Standard machine learning **libraries** were used:

- **NumPy & Pandas:** Data handling and manipulation
- **Scikit-learn:** Machine learning models, preprocessing, cross-validation
- **Matplotlib & Seaborn:** Visualization and result analysis

To guaranty consistency, we implemented a **shared utility code** that handled data loading, normalization, error computation, etc.

Input and output were **standardized** to zero mean and unit variance.

Models **trained** on normalized data. Mean Euclidean Error (**MEE**) computed in the **original** (unscaled) output space.

MODEL IMPLEMENTATION

- **Neural Network Architecture**: We explored architectures ranging from 1 to 3 hidden layers.
- **Other Model Architectures**: For SVM, we used an SVR model inside a MultiOutput Regressor wrapper, comparing Linear, Polynomial, and RBF kernels.

MODEL IMPLEMENTATION

NN Tuning, Initialization & Stop Conditions:

- **Regularization Technique:** L2 Regularization
- **Stop Conditions:** Implemented Early Stopping to stop training automatically when validation performance stopped improving, and set a hard upper limit of 2000 epochs as a fallback stop condition.
- **Initialization Schema:** Utilized a fixed random seed to initialize the weights. This ensures strict reproducibility of the results across runs.

NOVELTY

Since this is a Type B (**Comparison**) project, we didn't invent a **new algorithm**.

Our "novelty" is the **rigorous comparison framework** we used by implementing a **Grid Search with K-Fold CV**. This ensures our results were **statistically significant** and well-thought and not just **lucky guesses**.

MONK RESULTS

Task	Hyperparameters	MSE (TR)	MSE (TS)	Acc (TR)	Acc (TS)
MONK 1	(4, 4), $\alpha = 0.0$, $Sol = adam$	0.0024	0.0043	100.00%	100.00%
MONK 2	(4, 4), $\alpha = 0.0$, $Sol = adam$	0.0047	0.0062	100.00%	100.00%
MONK 3	(2, 2), $\alpha = 0.0001$, $sol = lbfgs$	0.0447	0.0328	95.08%	96.53%

Table 1: Neural Network (MLP) results on the MONK datasets

Task	Hyperparameters	MSE (TR)	MSE (TS)	Acc (TR)	Acc (TS)
MONK 1	$\eta = 0.1, \alpha = 0.01$	0.2153	0.2947	78.47%	70.53%
MONK 2	$\eta = 0.001, \alpha = 0.0001$	0.3846	0.3792	61.54%	62.08%
MONK 3	$\eta = 0.01, \alpha = 0.0001$	0.0656	0.0278	93.44%	97.22%

Table 2: Linear model results on the MONK datasets

Task	Hyperparameters	MSE (TR)	MSE (TS)	Acc (TR)	Acc (TS)
MONK 1	$K = 9$, W=Distance, $p = 1$	0.0234	0.1794	97.66%	82.06%
MONK 2	$K = 1$, W=Uniform, $p = 1$	0.0290	0.2794	97.10%	72.06%
MONK 3	$K = 3$, W=Distance, $p = 1$	0.0000	0.0919	100.0%	90.81%

Table 3: KNN results on the MONK datasets

Task	Hyperparameters	MSE (TR)	MSE (TS)	Acc (TR)	Acc (TS)
MONK 1	$C = 1$, ker=poly	0.0000	0.0000	100.0%	100.0%
MONK 2	$C = 10$, ker=poly	0.0450	0.2470	95.50%	75.30%
MONK 3	$C = 0.1$, ker=linear	0.0656	0.0278	93.44%	97.22%

Table 4: SVM model results on the MONK datasets

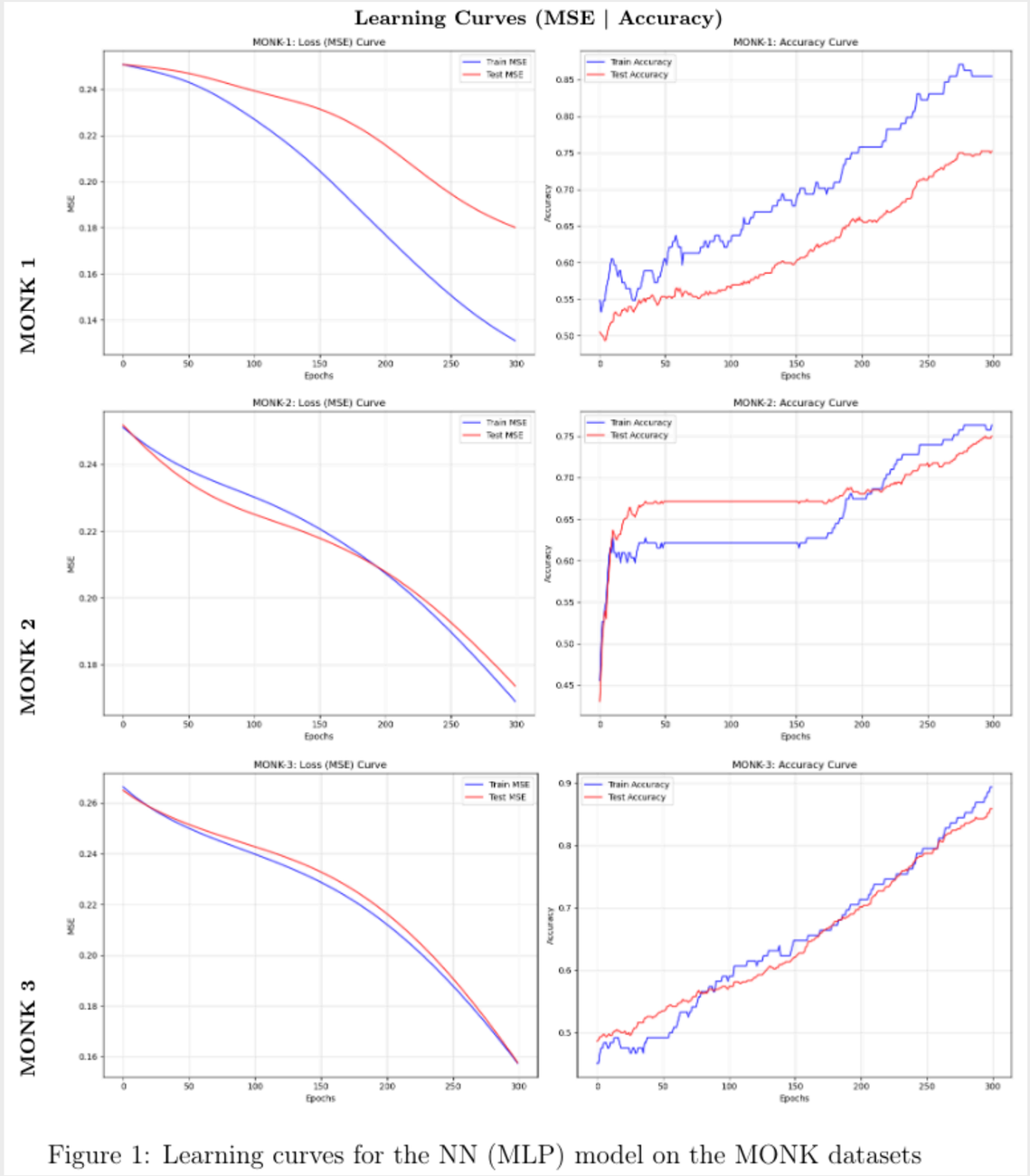

Figure 1: Learning curves for the NN (MLP) model on the MONK datasets

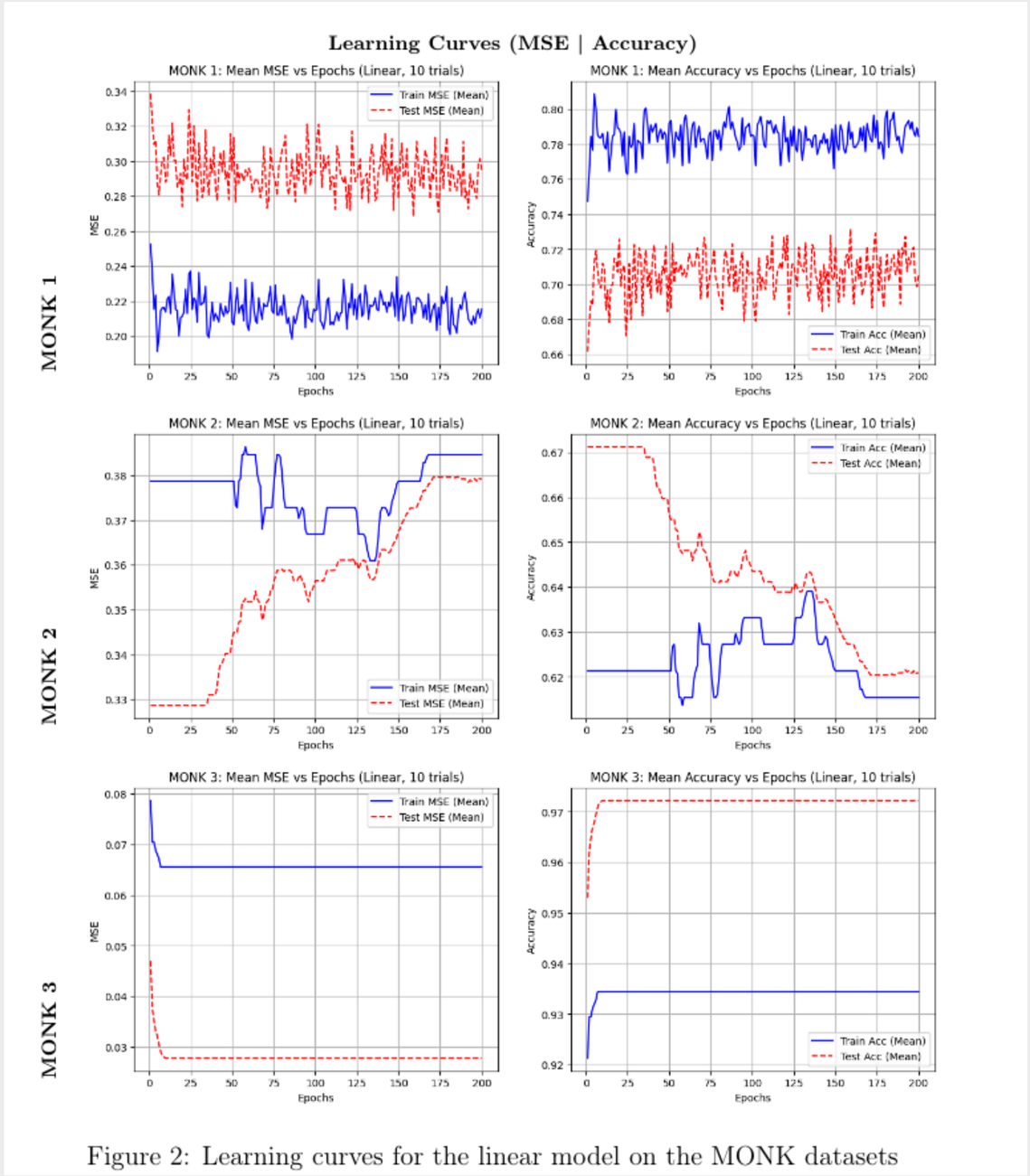

Figure 2: Learning curves for the linear model on the MONK datasets

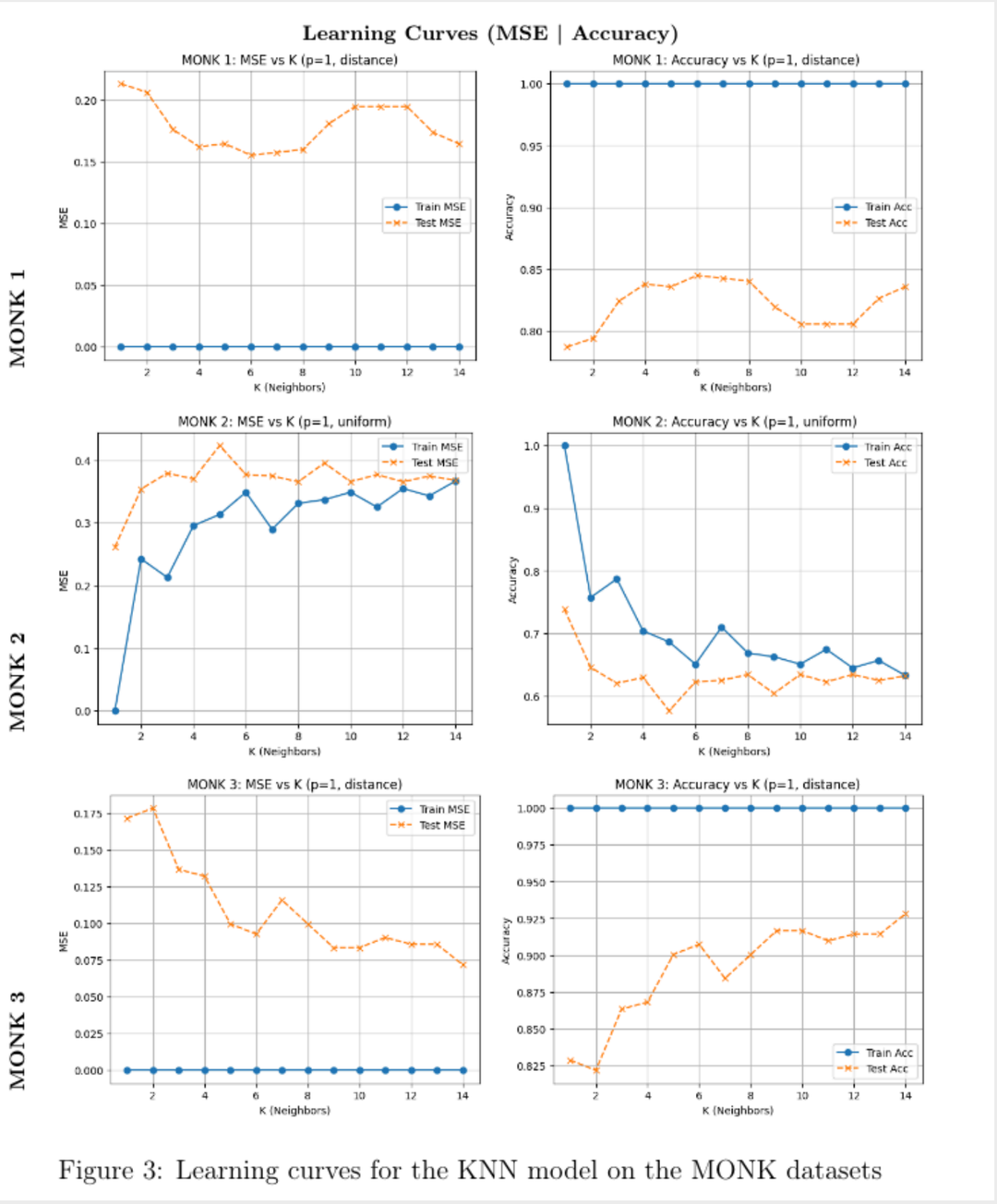

Figure 3: Learning curves for the KNN model on the MONK datasets

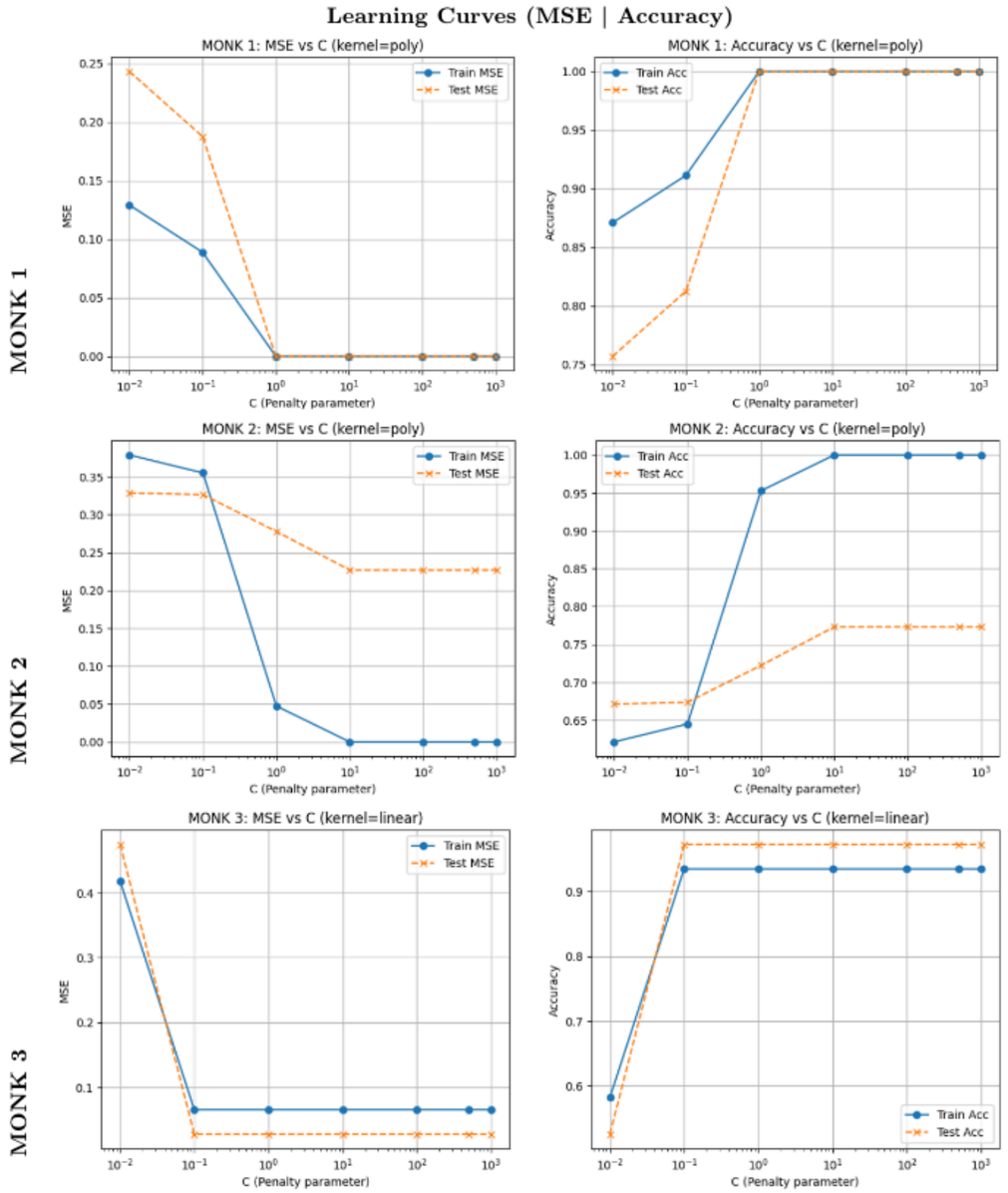

Figure 4: Learning curves for the SVM model on the MONK datasets

VALIDATION STRATEGY

DEVELOPMENT SET – 90%

- TR + VL
- Used for **training** and **hyperparameter tuning**

INTERNAL TEST SET – 10%

- Used only once for **final performance evaluation**
- Provides **unbiased estimate** of generalization

MODEL SELECTION

- Technique: **K-Fold Cross Validation (K=5).**
- Combined with **Grid Search** for **hyperparameter tuning**
- **Selection Criterion:**
  - **MONK:** Highest Average Validation Accuracy.
  - **CUP:** Lowest Average Validation MEE.

- Final Model Assessment
  - **Retrain** the best model on the entire **Dev Set (90%)**.
  - **Evaluate** once on the **Internal Test set (10%)**

HYPERPARAMETER SPACE

- **NN**
  - **Hidden Layer Sizes:** (50,), (20, 20), (40, 30), (50, 20), (70, 40), (100, 50), (30, 20, 10)
  - **Activation Function:** ‘relu’, ‘tanh’
  - **Optimization algorithm:** ‘adam’ (Mini-batch), ‘sgd’ Mini-batch, ‘lbfgs’ (Full-batch)
  - **Alpha (Regularization λ):** 0.0001, 0.001, 0.01, 0.1
  - **Learning Rate Init (η):** 0.001, 0.01, 0.1
  - **Momentum:** 0.5, 0.7, 0.9

- **SVM**
  - **Kernel**: Radial Basis Function (RBF), Linear, and Polynomial.
  - **C (Regularization)**: [0.1, 1, 10, 100].
  - **Epsilon (ϵ)**: [0.01, 0.1, 0.2].
  - **Gamma (γ)**: [Scale, 0.1, 0.01].

- **KNN**
  - **Number of neighbors**: K from 1 to 29
  - **Weighting strategy**: uniform, distance

CUP RESULTS

Best Model	MEE (VL)	MEE (TS)
Linear	25.41	26.67
NN	21.62	26.03
SVM	15.43	23.42
KNN	**16.44**	**15.13**

Observations:

- Linear model **underfits** (high bias)
- NN improved validation error but showed **generalization gap**
- SVM achieved best VL score but had **bad generalization**
- KNN showed **smallest generalization gap** and **best TS performance**

FINAL MODEL

Selected Model: **KNN**

Why?

- Lowest **Internal Test MEE** (15.13)
- **Very small gap** between Validation and Test
- More **stable generalization** compared to SVM and NN

Final Hyperparameters

- K = 3
- Weighting = distance
- Euclidean distance metric

LEARNING CURVE & MEE RESULTS

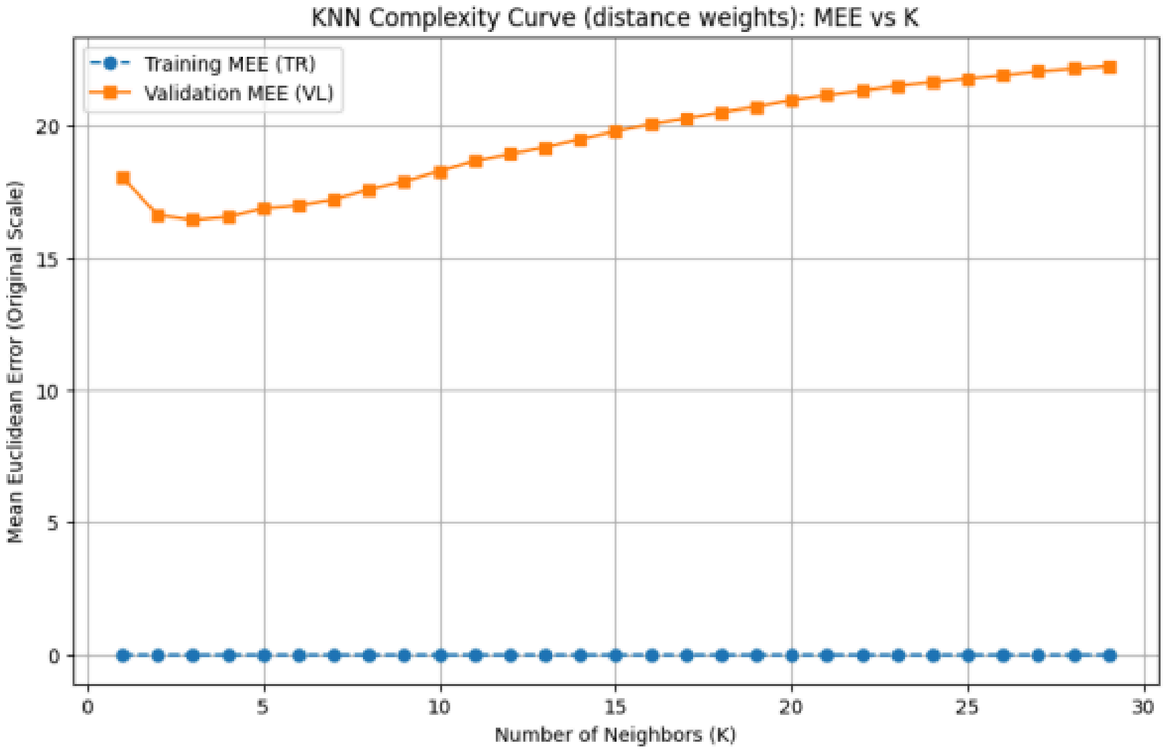

Figure 11: Learning Curve for KNN: MEE vs. Number of Neighbors (K)

MEE Results for the final model in the original scale:

- Model: **KNN (K=3, distance)**
- MEE (TR): **0.00**
- MEE (VL): **16.44**
- MEE (TS): **15.13**

Note: TR MEE = 0 due to distance weighting (perfect reconstruction of training samples).

DISCUSSION

What Did We Learn?

Model Complexity ≠ Better Performance

Although Neural Networks and SVM are more complex models:

- **SVM** achieved **VL MEE 15.43**, but **TS MEE 23.42**
- **NN** achieved **VL MEE 21.62**, but **TS MEE 26.03**

They both showed showed **overfitting tendency**

On the other hand, **KNN**, which is considered a **simpler model** compared to NN and SVM, achieved **better results** (VL MEE **16.44**, and TS MEE **15.13**)

DISCUSSION

Generalization Is the Key Factor

Our best model was KNN. Despite it not having the best **validation score**, it achieved the **best Internal Test performance**.

If our **model selection** was based only on best validation score, it would have been misleading, so **generalization (stability)** is more important.

University Of Pisa
Computer Science Department
Academic Year 2025/2026

THANK YOU

Authors: **Salmi, Chafai, Yakoubi**
Team: **SYC**